

WolfCafe - Use Case 9

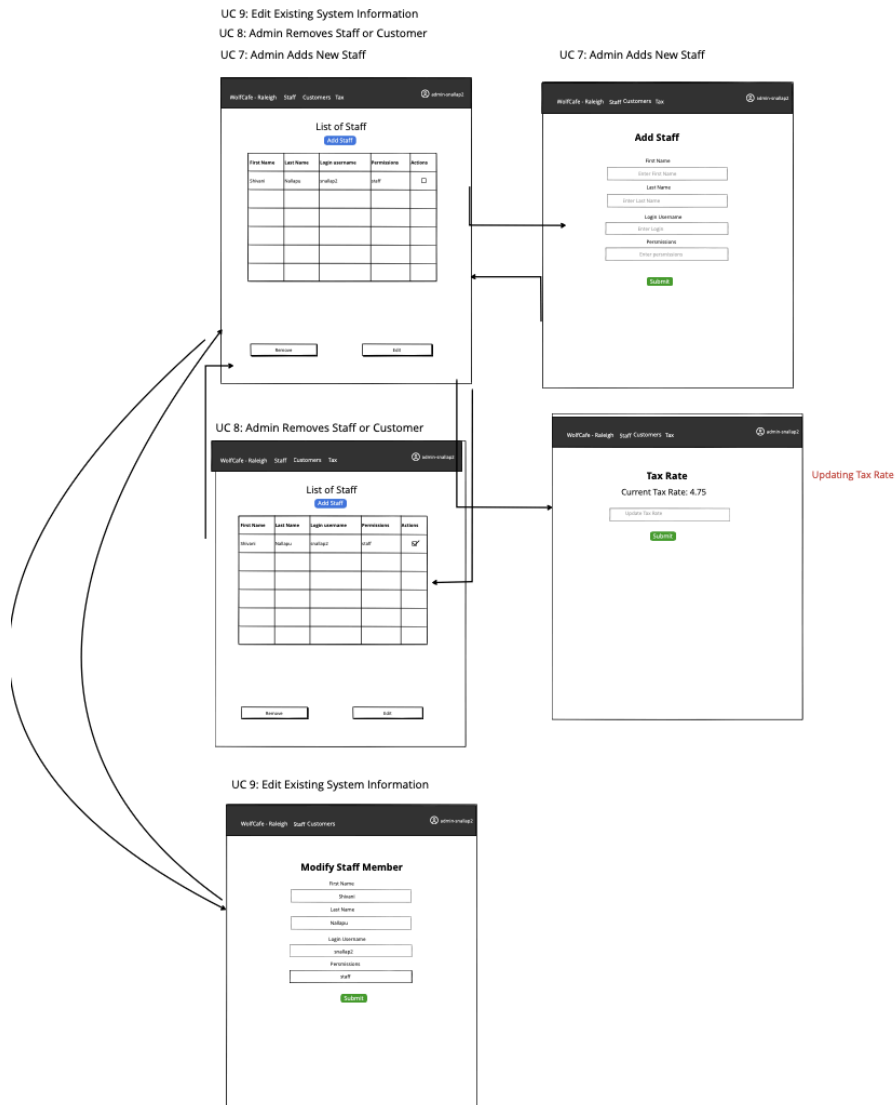
“Edit Existing System Information”

Design Decisions (frontend, backend) - done

Administration tasks such as the modification of crucial system information, like staff members and the tax rate for a given location. For our UML diagram, we considered new repositories to contain the role of a given user (**‘role repository’**) [**‘regular’**, **‘staff’**, **‘administrator’**] as well as needing a repository to store the users themselves (**‘user repository’**). We also needed to think about how we store the tax rate for each location. As we may have infinitely many locations, we create a DTO/Entity for each location (**‘Location’**, **‘LocationDTO’**), which stores the tax rate, as well as general info about each location (think address, # of staff, weekly sales, etc). We decided on restricting this information to only administrative users to follow the **principle of least knowledge**, as staff members and regular users don’t need to know these metrics or be able to modify the tax rate. We manage staff member modification through our authentication controller (denoted **‘AuthController’** class), which allows for the registration, handling of authentication and deleting of users. We manage modification of location details through a location controller (denoted **‘LocationController’** class). Adjacently, we also consider we need a service to handle more complex operations not suited for being included directly in both of the controller classes. We denote this as the **‘LocationControllerImpl’** and **‘AuthServiceImpl’** class, which contains several helper methods for the aforementioned class. We also had to consider how this would affect our frontend React components. We concluded on handling user/role data by utilizing React states, and having a singular frontend component (**‘StaffList.jsx’**) for management of staff members in a given location. Additionally, we needed a way in our front-end to manage location data for administrators, so we created another frontend component (**‘LocationList.jsx’**) for management of all locations. We concluded that for ease of use, administrators may need to sort through a long list of staff and locations depending on the size of a given location. For this reason, we implemented searching functionalities as well as a modular list tabular component for ease of view. This does affect our database, but given we are using **Hibernate** (our ORM tool) to map objects to our relational database (**MySQL**), our DTO (**‘UserDto’**, **‘RoleDto’**, **‘LocationDto’**) and entity objects (**‘User’**, **‘Role’**, **‘Location’**) allow us to easily manage these entries in our service implementations. In Hibernate, we will need many-to-one relationships as users will only have one role, and roles contain infinitely many users. Also, we have to consider that staff members can work at multiple locations, and administrators can handle infinitely many locations. To handle large lists of staff and location management, we introduce our decision of **server-side pagination** to improve experience for administration, improving server responsiveness. Testing will be done using the **Selenium** framework on the front-end, with **JUnit** being used in the backend to test our repositories, controllers and services.

Frontend Elements

Original Design (**SAME AS UPDATED DESIGN**)

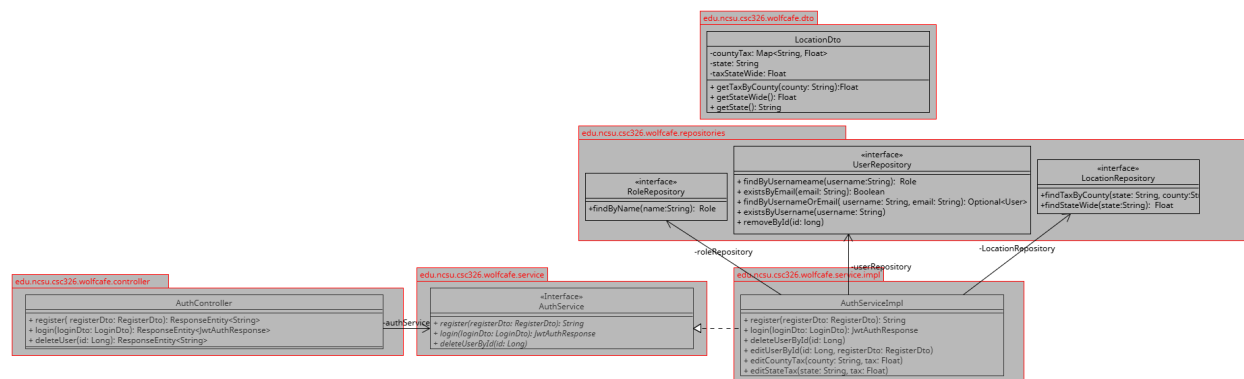


Changelog

- Since it's iteration 1A we did not change anything

Backend Elements

Class Diagrams -



Original Design (**SAME AS UPDATED DESIGN**)

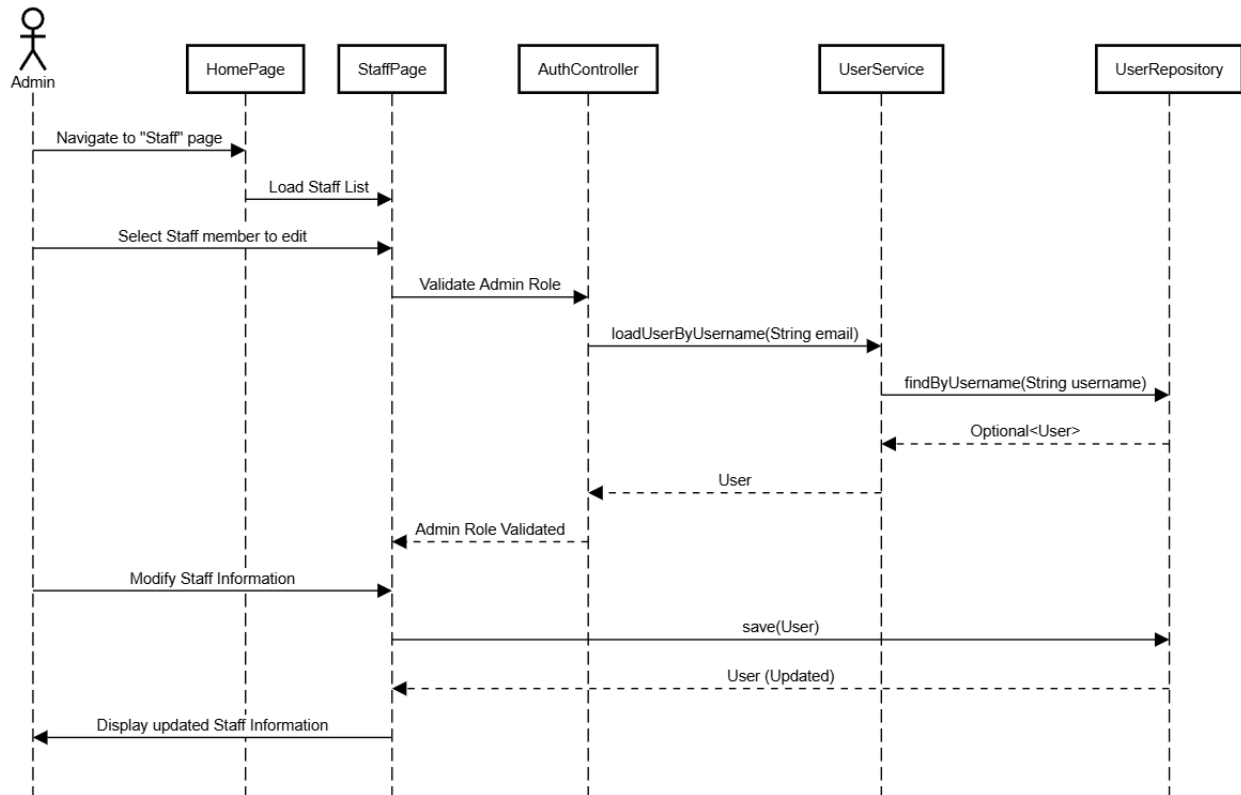
Changelog

- Since it's iteration 1A we did not change anything

Sequence Diagram:

Original Design (**SAME AS UPDATED DESIGN**)

Edit Existing System Information



Changelog

- Since it's iteration 1A we did not change anything